

Training an IT Admin Chatbot Using a Vector Database (Qdrant)

This document explains how to train and operate an IT Administrator chatbot (Terry) using a vector database. The chatbot relies on semantic similarity to infer user intent and route requests to predefined IT workflows safely and accurately.

1. Why Use a Vector Database?

Traditional keyword-based matching is fragile. A vector database allows the chatbot to understand user intent even when phrasing varies. Qdrant stores embeddings of example queries and retrieves the closest match based on semantic similarity.

2. Supported Capabilities of the Chatbot

- 1 Web server and NGINX status checks
- 2 Machine (VM/host) UP or DOWN checks
- 3 SFTP operations (list, upload, download)
- 4 Local file listing
- 5 Samba server operations
- 6 NTP sync and NTP client checks

3. Vector Database Structure (Qdrant)

Each supported intent is represented by multiple example phrases. Each phrase is embedded into a vector and stored in Qdrant with metadata (payload) describing the intent and the tool to be used.

Payload Fields

- 1 intent_id – unique identifier for the intent
- 2 intent_name – human-readable intent name
- 3 tool – the tool that must be used
- 4 functionality – description of the action
- 5 confidence_required – minimum similarity threshold

4. Example Training Phrases

Each of the following sentences should be embedded and stored as a separate vector:

- 1 Check if the website is running
- 2 Is nginx active
- 3 Check all machines
- 4 Are any hosts down
- 5 List files on the SFTP server

- 6 Upload a file to SFTP
- 7 Show local files
- 8 List shared VM files
- 9 Is NTP synced
- 10 Show NTP clients

5. Intent Inference Workflow

1. User submits a message.
2. The message is converted into an embedding.
3. Qdrant is queried for the nearest vector.
4. If similarity score meets the confidence threshold, the intent is accepted.
5. The chatbot routes execution to the correct tool.
6. If confidence is low or intent unsupported, the chatbot responds with 'I don't know'.

6. Safety and Guardrails

The chatbot must never guess or perform actions outside its defined scope. Vector similarity controls execution, ensuring that only high-confidence requests trigger real IT operations.

Conclusion

By combining a vector database with strict tool routing and confidence thresholds, the chatbot becomes reliable, safe, and production-ready for IT administration tasks.